

GT Assure

AI-assisted control testing and PBC management for an assurance practice. Everything runs on the consultant's machine; no client evidence leaves it.

Control testing is the most time-consuming activity on an engagement: requesting documentation, chasing what has not arrived, reading large volumes of evidence, and judging whether it satisfies a control description. GT Assure compresses the mechanical part — retrieval, first-pass assessment, citation, PBC administration — and leaves the judgement with a named human who approves every conclusion before it can be exported.

Five constraints came before any code, and the architecture is built around them rather than aspiring to them: fully local inference, framework-agnostic control definitions, a human in the loop, an append-only audit log, a pinned model per run, and synthetic data only so the demo never needs a real client.

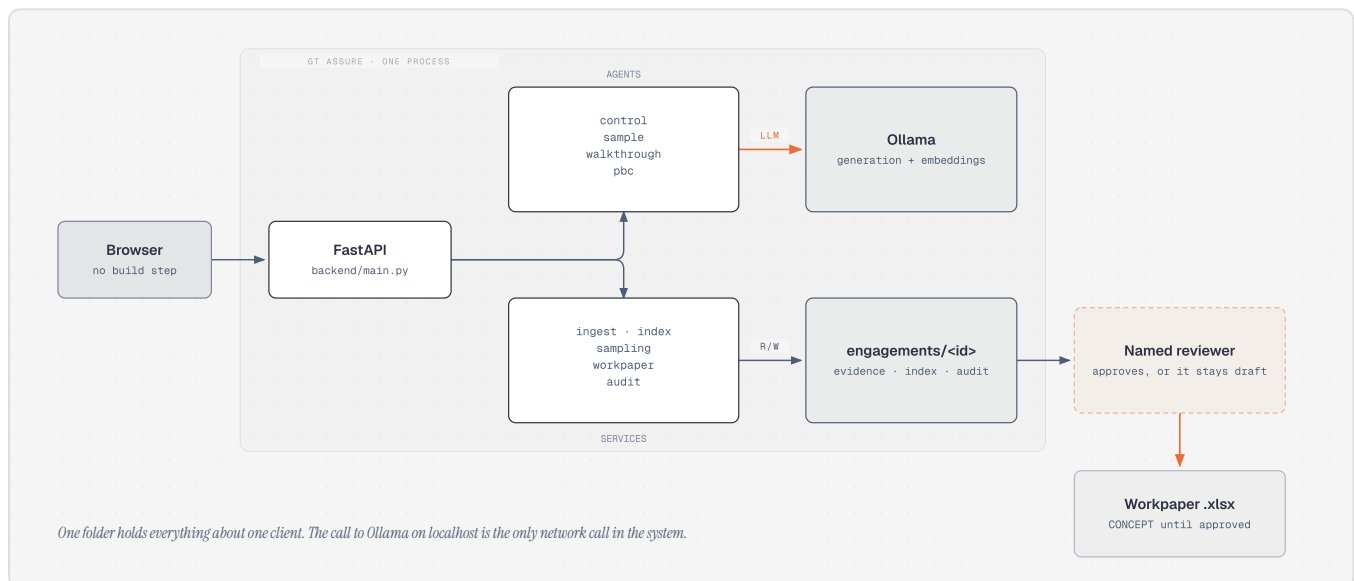


Fig. 1 – one folder per client; one network call in the system.

What the model decides, and what code decides

The division of labour is the thing I would show a partner. Per control, the model does two jobs: turn a control description into acceptance criteria and retrieval probes, then judge the retrieved evidence against those criteria while citing the numbered excerpts it used. Everything else is code.

Two of those steps exist because of a specific failure. Retrieval returns whole documents in page order, not the closest paragraph, because a control is evidenced by a document: the incident plan's severity matrix and its named crisis

roles sit on different pages, and judging the control on whichever page embedded closest manufactures false gaps. And if retrieval returns nothing, the second model call never happens at all — asking a model to assess a control with no evidence in front of it is an invitation to answer from general knowledge about what such a control usually looks like.

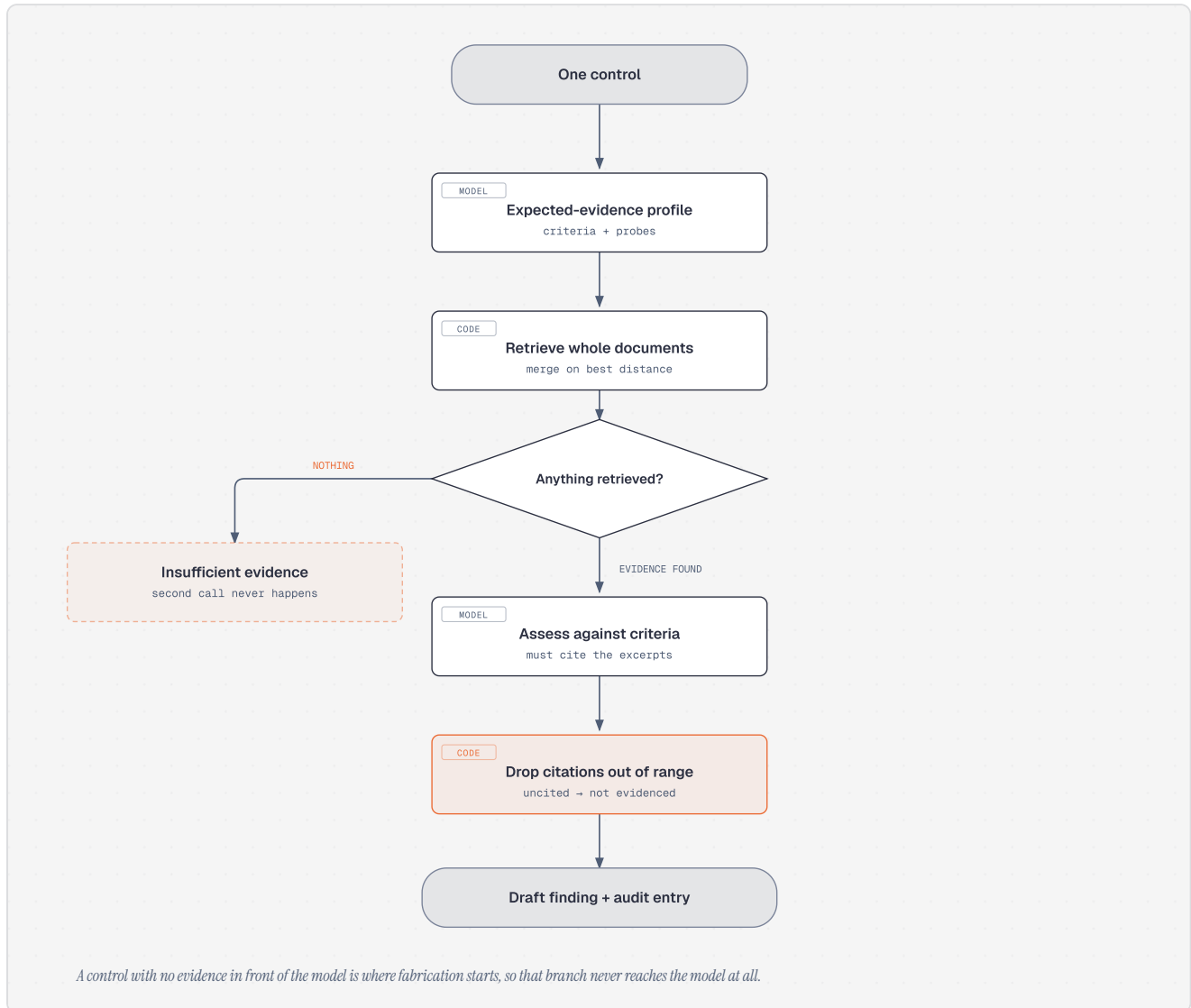


Fig. 2 – the per-control pipeline. The empty branch never reaches the model.

Sample testing, where most of the real effort is

Assessing an evidence pack is design-level work. Most of the hours on a real engagement go into the next step: drawing a sample from the client's population and testing each item against named attributes. GT Assure does that too, and again the split is the point.

STEP	WHO
Sample size	a lookup in the firm's table (methodology/sampling.yaml)
Which items are drawn	a seeded draw, reproducible from engagement + control + population hash
Which attributes exist	the framework file
Whether evidence corroborates an item	the local model — one call per item
Exceptions, conclusion, workpaper	code

Three interlocks stand between the model and the filed workpaper, and each exists because an 8B did the thing it prevents on this demo pack. Only passages that *literally* name the item are shown to the model — on the first live run it concluded all twelve items were established, citing September release documentation for a February change. A conclusion of `vastgesteld` without a resolvable citation is downgraded. And `n.v.t.` is refused where the client's own export asserts the attribute, because "not applicable" is how real gaps quietly stay out of the exception count.

Evidence is found literally rather than semantically, which was a correction: "which passages mention ticket 14721" is a text question, and a ticket number carries almost no embedding signal, so the correct document repeatedly fell outside top-k while unrelated ones ranked above it.

Measured, and I would quote it that way

On `qwen3:8b`, across two runs of the twelve-item demo sample: the changes absent from the release documentation concluded *niet vastgesteld* on every attribute in both runs, and CAB-approval and test-environment attributes were stable. The acceptance attribute on cleanly documented items was **not** stable — four of eight were flagged as exceptions in one run that should have passed.

The errors run conservative: the tool over-refers to the reviewer rather than passing items silently. That is the right direction for assurance work, and it is not accuracy. The same holds at design level: a control with no evidence at all is reliably called *insufficient evidence* by a 14B, while the 8B is more likely to accept an adjacent document. The deterministic grounding checks catch the uncited case; they cannot catch a wrong-but-cited one, which is exactly what the reviewer is for.

Pitch the cited reasoning, not the label. That is the whole demo instruction and it is also the honest description of what an 8B is good for.

Walkthroughs and the anti-fabrication interlock

The Proces tab of a workpaper is written from a walkthrough interview. Paste the transcript and GT Assure produces the process description and a flowchart. The transcript is numbered by line and **every extracted step must cite the line it came from**; a step that cites nothing is discarded and counted. That is exact rather than fuzzy on purpose — a line number either resolves to something somebody said or it does not. Asking for a verbatim quote instead would have been weaker in both directions, because checking a paraphrase needs a similarity threshold and an 8B paraphrases constantly.

Control references in the workpaper are whitelisted against the control's own identifiers, because an invented reference in that column reads as firm methodology. Steps are renumbered by position, because the model's own numbering drifts. The flowchart is laid out deterministically, so the same steps always render byte-identical SVG and two workpapers for the same process can be compared.

What I would want a reviewer to push on

- Approval is enforced by writing the export file only at approval time. It is elegant, and it means the guarantee lives in a file's existence rather than in a check I can point at.
- The 8B's attribute-level instability is measured but not bounded. I do not know how it behaves on a real population an order of magnitude larger.
- The audit chain is append-only JSONL on the same disk as the evidence. Tamper-evident, not tamper-proof.